

Cassava Leaf Disease Classification: A Comparative Study of Classical Machine Learning and Deep Learning Approaches

Samuel Kwizera Ihimbazwe

Introduction to Machine Learning — Summative Project
African Leadership University

GitHub: <https://github.com/Samkwizera/Model-Training-and-Evaluation.git>

Demo Video: <https://youtu.be/MVyEAzU1XwY>

Collab notebook:  `SamuelkwizeraIhimbazwe-cassava_disease_classification.ipynb`

Abstract

This project compares classical machine learning and deep learning approaches for cassava leaf disease classification on 21,397 field images across five categories. I evaluated three classical models (Logistic Regression, Random Forest, and an RBF-kernel Support Vector Machine) against a four-block Sequential CNN and a fine-tuned MobileNetV2. The dataset has a $12.1\times$ class imbalance, so I used macro-averaged F1 rather than accuracy as the main metric. The augmented Sequential CNN came out as the strongest (test macro-F1 = 0.588, AUC = 0.912), narrowly beating fine-tuned MobileNetV2 (0.509). That ranking was not what I expected, and it traces back to a distribution shift between the two transfer learning phases. Among the classical models, the SVM led (macro-F1 = 0.494). Random Forest collapsed to majority class predictions (0.154) because PCA diluted the MobileNetV2 embeddings, which a variance analysis showed carry 93.0% of the raw feature variance, against 7.0% for HOG and roughly 0.0% for colour histograms. Three things stand out from the results: macro-F1 is far more informative than accuracy under imbalance, augmentation improves generalisation even with the architecture held constant, and keeping the data regime consistent across training phases can matter more than the head start that pretraining is supposed to give.

1. Introduction

Cassava is actually the second largest source of dietary carbohydrates in sub-Saharan Africa, feeding more than 800 million people. In East Africa, it is a food security crop that smallholder farmers depend on during drought, yet four diseases threaten it: Cassava Bacterial Blight (CBB), Cassava Brown Streak Disease (CBSD), Cassava Green Mottle (CGM), and Cassava Mosaic Disease (CMD). Together they cause yield losses of 15 to 24% a year. Diagnosis still depends on extension officers walking the fields, which is slow and only reaches so far. Cheap smartphones and modern computer vision change that calculus. A farmer can photograph a leaf and get an answer in seconds, which cuts the delay between infection and treatment.

I treat diagnosis as a five class image classification task and compare two families of methods on the same data. The first is a classical pipeline that feeds handcrafted and pretrained features into Scikit-learn classifiers. The second uses end-to-end TensorFlow models: a CNN trained from scratch and a transfer learning model on MobileNetV2 [6]. The main question is which family generalises better to unseen field images, and why. Two smaller questions sit underneath it. How much does augmentation help when the architecture stays fixed, and does ImageNet pretraining actually give a reliable advantage here? The data is the Kaggle Cassava Leaf Disease Classification set from the Makerere AI Research Lab in Uganda, 21,397 labelled field photographs. It is one of the more realistic public benchmarks for this problem because the photos were taken in real farming conditions rather than a lab. The project contributes a controlled eight experiment comparison across both families, a diagnostic account of why an ensemble method failed where a kernel method succeeded, and evidence that keeping the training regime consistent can outweigh the benefit of transfer learning.

2. Literature Review

Automated plant disease detection has grown quickly since the mid-2010s. Hughes and Salathé [12] released the PlantVillage dataset (54,000+ images, 26 diseases, 14 crops), which kicked off a wave of deep learning work in agricultural imaging. Its weakness is that the images were shot in controlled lab conditions, so models trained on it tend not to hold up in the field. That gap is exactly why this study uses field collected images. Mohanty et al. [4] applied deep CNNs to PlantVillage at scale and reached 99.35% accuracy with AlexNet and GoogLeNet, but the same models did poorly on actual field photographs for that lab to field reason. Ramcharan et al. [5] tackled the gap head on with a MobileNet classifier trained on smartphone images from Ugandan farms, essentially the same population as this dataset, and reported 93% accuracy. That figure is a useful reference point for how hard the problem is, and it is why I reached for MobileNetV2 in the transfer learning experiments. Mwebaze and Owomugisha [3] did foundational work on plant disease ML in the African context and released the competition dataset. Two broader surveys back up the trend. Kamilaris and Prenafeta-Boldú [14] looked across 40 studies and found CNNs consistently beat classical methods but needed large labelled sets, and Ferentinos [15] reported 99.5% in distribution accuracy that dropped on field photographs.

On the classical side, support vector machines have a solid track record on image classification. Cortes and Vapnik built the maximum margin formulation [7], and the kernel trick [8] extends it to nonlinear problems. SVMs do well when the features are compact and well structured, which is the case here once the MobileNetV2 embeddings dominate. Breiman's Random Forest [9] captures nonlinear interactions without a kernel, but it is sensitive to feature quality when dimensionality reduction blends very different

feature types, and that sensitivity turns out to matter in Section 6. The Histogram of Oriented Gradients descriptor [13] is a standard texture feature; I included it for its sensitivity to edge direction, though the variance analysis later shows its actual contribution is small.

Class imbalance is a well-studied problem, and it is severe here: CMD is 61.5% of the images and CBB only 5.1%. Buda et al. [10] found that combining a class-weighted loss with augmentation beats either one on its own, And this is reason which is why I use balanced weighting alongside augmentation. Shorten and Khoshgoftaar [11] surveyed more than 60 augmentation techniques and concluded that geometric and colour-space transformations work best for natural images, and those are exactly the operations in my pipeline. One question the literature leaves open is the one this project goes after: on a single realistic field dataset, under one preprocessing and evaluation protocol, does transfer learning reliably beat a from scratch CNN, and does an ensemble stay strong once heterogeneous features are squeezed through PCA? The experiments below answer both.

3. Methodology

3.1 Dataset Description

The dataset has 21,397 JPEG images, each labelled CBB (class 0), CBSD (1), CGM (2), CMD (3), or Healthy (4). A 1,000-image audit confirmed they are all 800×600 pixels with no corrupt files. The class split is badly skewed: CMD is 61.5% of the images and CBB only 5.1%, a 12.1× ratio between the largest and smallest class (Figure 1). This skew is the reason accuracy is a poor metric here. A model that simply guesses CMD every time would score 61.5% accuracy without learning anything, so I report macro averaged F1 throughout instead.

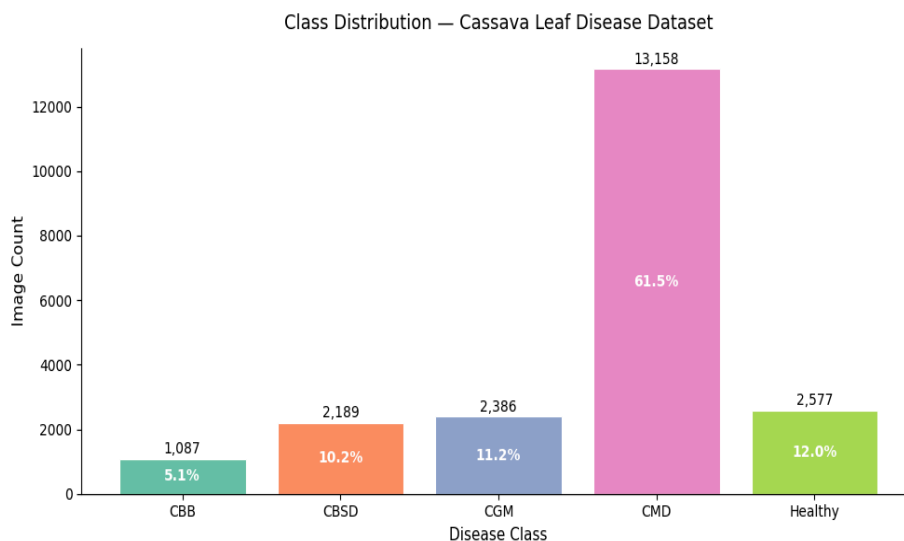


Figure 1. Class distribution of the Cassava Leaf Disease dataset. CMD dominates with 61.5% of images while CBB accounts for only 5.1%, yielding a 12.1× imbalance ratio that motivates the use of macro-averaged F1.

3.2 Exploratory Data Analysis

Three random samples per class (Figure 2) show why the task is hard. Several classes share similar symptoms, and the generalised yellowing of CGM and CMD is the clearest example. CBB’s water-soaked

spotting can also pass for ordinary blemishes on a healthy leaf. The backgrounds are full of soil, hands, and neighbouring foliage, which is what real field capture looks like. A dimension audit on 500 images (Figure 3) shows the sizes cluster tightly around 800×600, so resizing is straightforward.

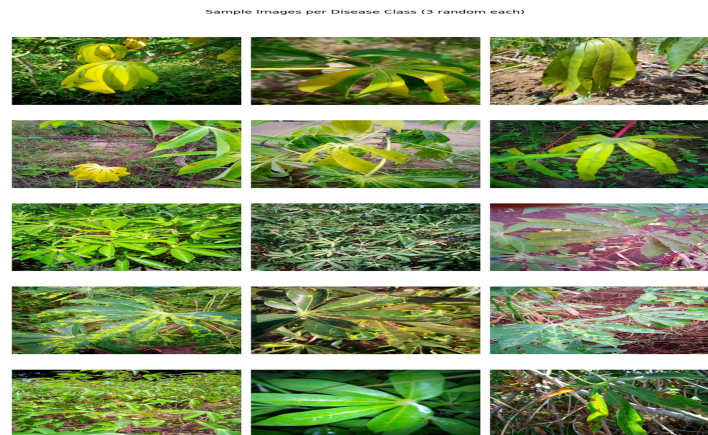


Figure 2. Three random sample images from each of the five classes. Note the visual similarity between CGM and CMD (generalised yellowing) and the cluttered field backgrounds typical of smartphone capture.

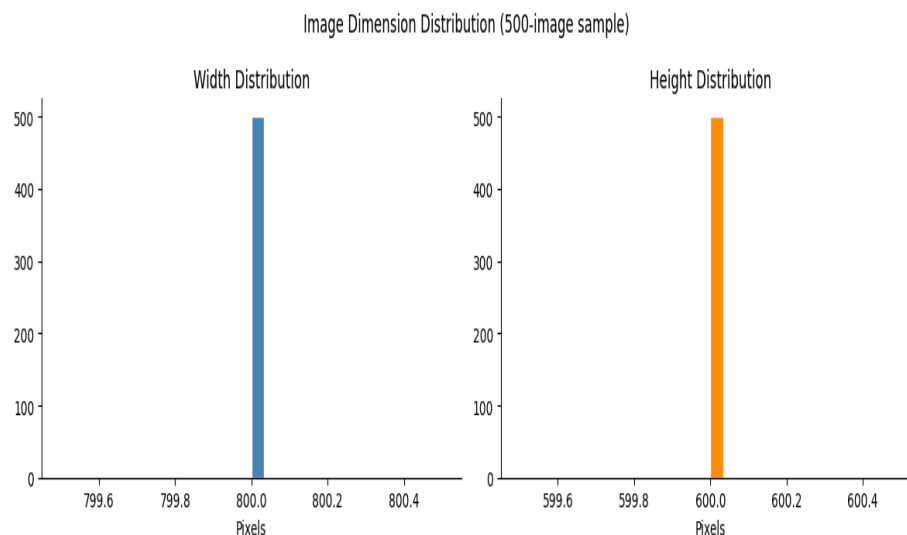


Figure 3. Width and height distributions across a 500-image sample. Dimensions are tightly clustered around 800×600 pixels, indicating consistent capture resolution.

3.3 Data Preprocessing

A stratified 70/15/15 split gave training (14,977), validation (3,210), and test (3,210) partitions while keeping the class ratios intact, which matters when the smallest class has fewer than 1,100 training examples. Class weights came from the training split alone (CBB 3.936, CMD 0.325) and were applied to every model. For the classical pipeline I decoded images with OpenCV, resized them to 224×224, standardised with a StandardScaler fitted only on the training split, then reduced the result by PCA from 7,460 to 512 dimensions. For the deep learning pipeline I decoded with tf.io, resized to 224×224, and normalised to [0, 1]. Augmentation was applied to training batches only: random flips, rotations up to 20°, zoom up to 15%, and brightness and contrast adjustment. BUT One detail caused me some real trouble.

The brightness layer's `value_range` had to be set to (0.0, 1.0), because TensorFlow 2.20's `RandomBrightness` defaults to a [0, 255] scale, which corrupts [0, 1] normalised images and made training fail in my earlier runs.

3.4 Classical Machine Learning Pipeline

I concatenated three feature descriptors per image. The first is an HSV colour histogram (32 bins per channel, 96 dimensions); I chose HSV over RGB because it keeps hue separate from luminance. The second is a HOG descriptor [13] (9 orientations, 16×16 cells, 2-cell blocks, 6,084 dimensions). The third is a frozen ImageNet MobileNetV2 [6] with global average pooling, giving a 1,280-dimensional embedding. The combined 7,460-dimensional vector was standardised and then compressed by PCA to 512 components.

A variance analysis turned up a lopsided picture of how useful each feature group actually is. The MobileNetV2 embeddings account for 93.0% of the raw variance, HOG for 7.0%, and the colour histograms for roughly 0.0%. The PCA scree plot (Figure 4) shows that 512 components keep only 68.5% of the variance, short of the 95% I was aiming for. The reason is HOG: its 6,084 near-zero-variance dimensions drag the cumulative curve up slowly and crowd out the more useful CNN features. I then trained three classifiers on the compressed features: Logistic Regression at two L2 strengths ($C = 0.01$ and $C = 1.0$), Random Forest (300 trees, `balanced_subsample`), and an RBF SVM ($C = 10$, `gamma = scale`, Platt scaling). All of them used balanced class weighting.

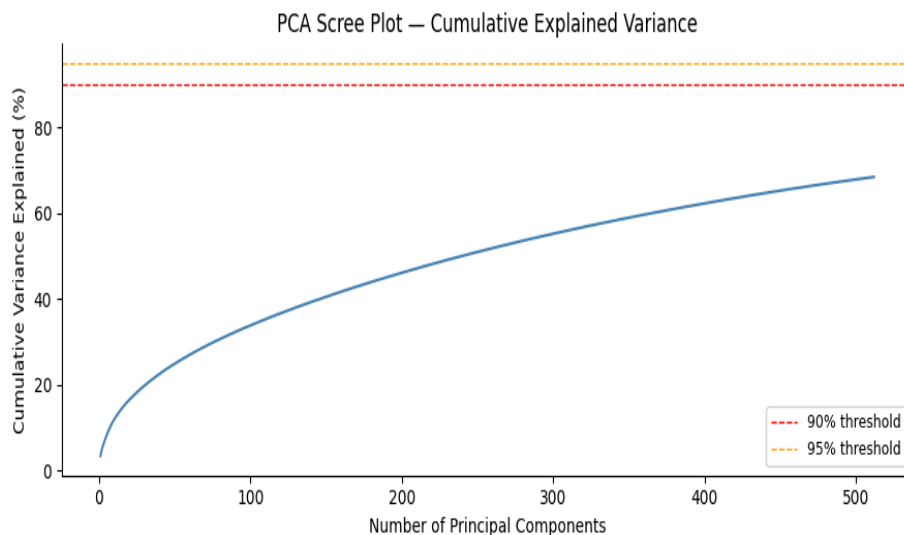


Figure 4. PCA scree plot showing cumulative explained variance against the number of components. The 512 retained components capture only 68.5% of variance, a direct consequence of HOG's many low-variance dimensions.

3.5 Deep Learning Pipeline

The Sequential CNN has four convolutional blocks (Conv2D with 32/64/128/256 filters, each followed by BatchNormalization and MaxPooling2D), then GlobalAveragePooling2D, Dense(256) with ReLU, Dropout(0.5), and a softmax output. I trained it with Adam at 1e-3 alongside `ReduceLROnPlateau` and `EarlyStopping`. One run used clean images as an overfitting baseline and another used the augmented pipeline. The transfer-learning model used MobileNetV2 (ImageNet weights, no top) through the Functional API, with an in model layer mapping [0, 1] inputs to [-1, 1] via $x = \text{inputs} \times 2 - 1$ in place of

the standard preprocess input. Training ran in two phases. Phase 1 froze the backbone and trained only the head on clean images. Phase 2 unfroze the top 30 of 154 layers and fine-tuned on augmented data at a $100\times$ lower rate (Adam, $1e-5$) to avoid catastrophic forgetting. Figure 5 shows one augmented batch and confirms the transformations behave correctly after the value range fix.



Figure 5. One batch of augmented training samples. Random flips, rotations, zoom, brightness, and contrast adjustments are visible, and all pixel values remain correctly within the $[0, 1]$ range.

4. Experiments

I ran eight experiments, four per family, with each one building on the last. Experiments 1 and 2 vary regularisation in Logistic Regression. Experiment 3 swaps in an ensemble. Experiment 4 introduces a kernel. Experiments 5 and 6 isolate augmentation in a from-scratch CNN, and Experiments 7 and 8 are the two transfer-learning phases. Table 1 records validation and test metrics for all eight runs. Because the CNN and transfer phases share checkpoints, the final test evaluation reports five distinct models, the best of each approach.

Table 1. Complete experiment log. Validation metrics are reported for classical models (selected on the validation split) and final test metrics for the five best models. Dashes indicate metrics not separately evaluated at that stage.

Exp	Model	Key Hyperparameters	Val Acc	Val F1	Test Acc	Test F1	Test AUC
1	Logistic Regression	C=0.01, balanced, PCA-512	0.6084	0.4743	0.6168	0.4856	0.8378
2	Logistic Regression	C=1.0, balanced, PCA-512	0.6084	0.4735	0.6168	0.4856	0.8378
3	Random Forest	n=300, bal_subsample, PCA-512	0.6150	0.1524	0.6153	0.1535	0.7778
4	SVM (RBF)	C=10, gamma=scale, PCA-512	0.7202	0.4979	0.7137	0.4941	0.8676
5	Sequential CNN	No aug, Adam($1e-3$), Drop(0.5)	0.7769	0.6503	0.7396	0.5877	0.9120
6	Sequential CNN	Aug, Adam($1e-3$), Drop(0.5)	0.7268	0.5782	0.7396	0.5877	0.9120
7	MobileNetV2 head	Frozen, no aug, Adam($1e-3$)	0.7044	0.5768			

Exp	Model	Key Hyperparameters	Val Acc	Val F1	Test Acc	Test F1	Test AUC
8	MobileNetV2 FT	Top-30 unfrozen, aug, Adam(1e-5)	0.5975	0.5171	0.5835	0.5089	0.8693

The progression is deliberate. Raising C in Logistic Regression barely moves the numbers, which tells me the feature representation, not regularisation, is the binding constraint. Random Forest then drops macro-F1 sharply, which is what pushed me toward a kernel method, and the kernel restores it. On the deep learning side, augmentation trades a small dip in validation F1 for a stronger and steadier test result, and the two-phase transfer experiment is where the distribution shift discussed in Section 6 shows up.

Table 2. Per-class F1 score on the test set for the five best models. CMD is well handled by every model, whereas CBB and CGM remain the hardest classes.

Model	CBB	CBSD	CGM	CMD	Healthy
LR ($C=1$)	0.367	0.429	0.404	0.786	0.443
Random Forest	0.000	0.000	0.000	0.762	0.005
SVM (RBF)	0.358	0.440	0.342	0.857	0.474
CNN (aug)	0.441	0.550	0.565	0.899	0.484
MobileNetV2 FT	0.458	0.491	0.429	0.714	0.453

5. Results

This section walks through the evaluation figures and ties each one back to performance. Figure 6 gives the overall comparison, plotting test accuracy, macro-F1, and macro AUC across the five final models. The Sequential CNN leads on every metric and the SVM is the best classical model. Random Forest is the odd one out, a clear outlier on macro-F1 despite a respectable AUC, which I unpack below.

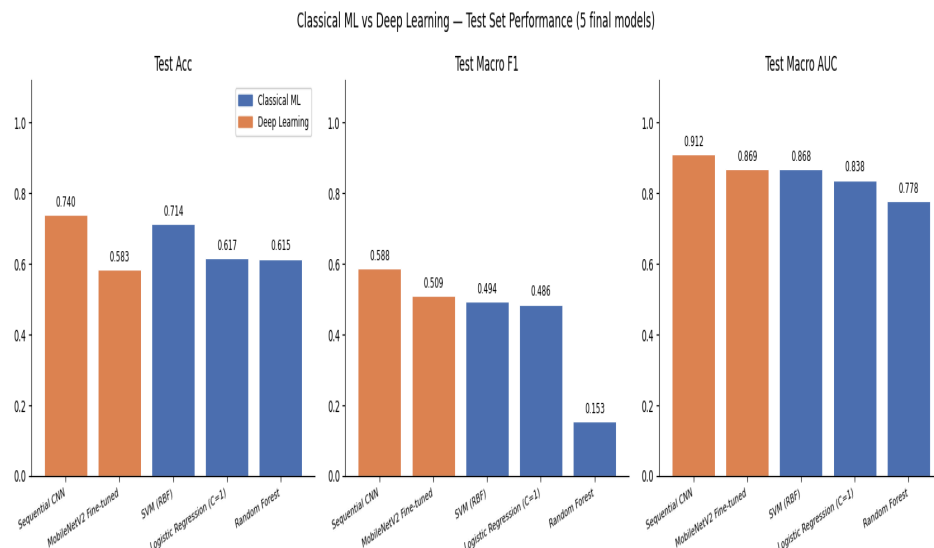


Figure 6. Classical ML versus deep learning on the test set across accuracy, macro-F1, and macro AUC. The augmented Sequential CNN (orange) leads on all three metrics; Random Forest’s collapse on macro-F1 stands out against its moderate AUC.

The validation confusion matrices for Random Forest (Figure 7) and SVM (Figure 8) make the classical failure and success concrete. Random Forest dumps almost every sample into CMD, so its CMD recall is near perfect and its recall everywhere else is close to zero. The SVM spreads its predictions out, recovers usable recall for CBSD and Healthy, and still trips up on CGM and CBB by confusing them with CMD.

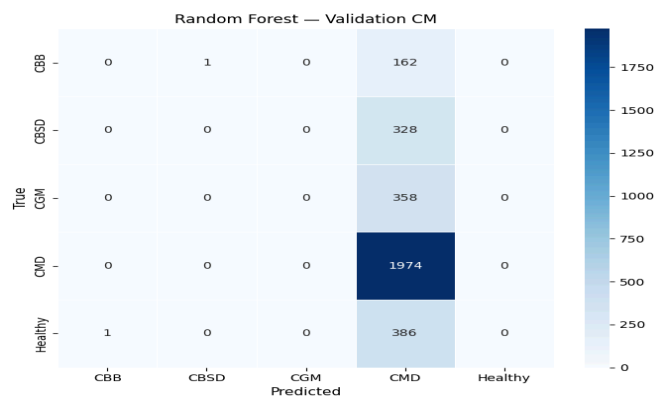


Figure 7. Random Forest validation confusion matrix. Predictions concentrate on CMD, the majority class, producing zero recall for CBB, CBSD, CGM, and Healthy.

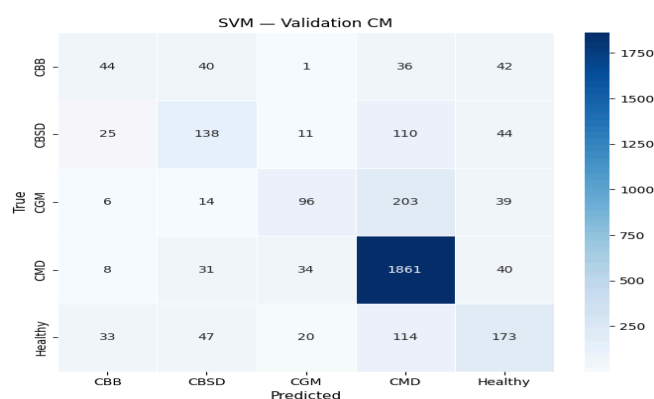


Figure 8. SVM (RBF) validation confusion matrix. Predictions are distributed across all classes, with the main residual confusion between CGM/CBB and the dominant CMD class.

The learning curves diagnose what the deep learning models are doing. The non-augmented CNN (Figure 9) is a textbook high variance case: training accuracy climbs steadily while validation accuracy bounces around between roughly 0.08 and 0.73. Augmentation (Figure 10) smooths that out. Validation accuracy is still noisy because the inputs are now harder, but it reaches higher peaks and holds them, and the gap between training and validation narrows. The transfer curves (Figure 11) tell the opposite story. Phase 1 converges cleanly to 0.704, then Phase 2 drops straight to 0.585 and never climbs back, which triggers early stopping.

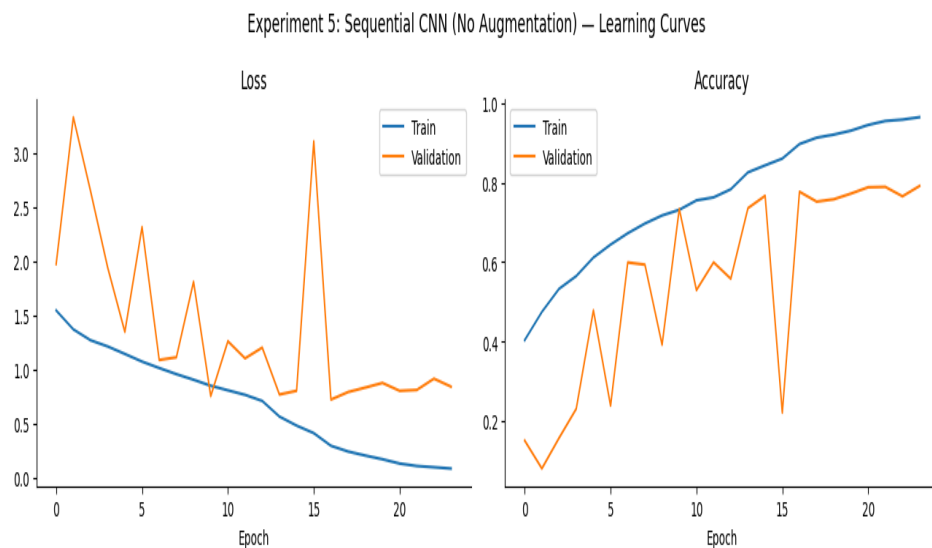


Figure 9. Learning curves for the non-augmented Sequential CNN. The wide oscillation of validation accuracy against steadily rising training accuracy is the classic signature of high-variance overfitting.

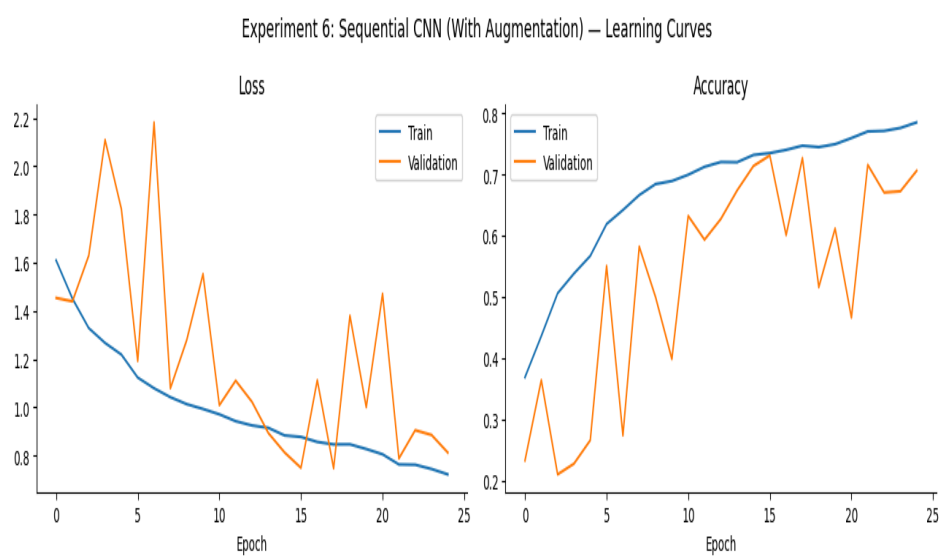


Figure 10. Learning curves for the augmented Sequential CNN. Augmentation stabilises validation performance and narrows the train/validation gap relative to Figure 9.

Experiments 7 & 8: MobileNetV2 — Learning Curves (Phase 1 + 2)

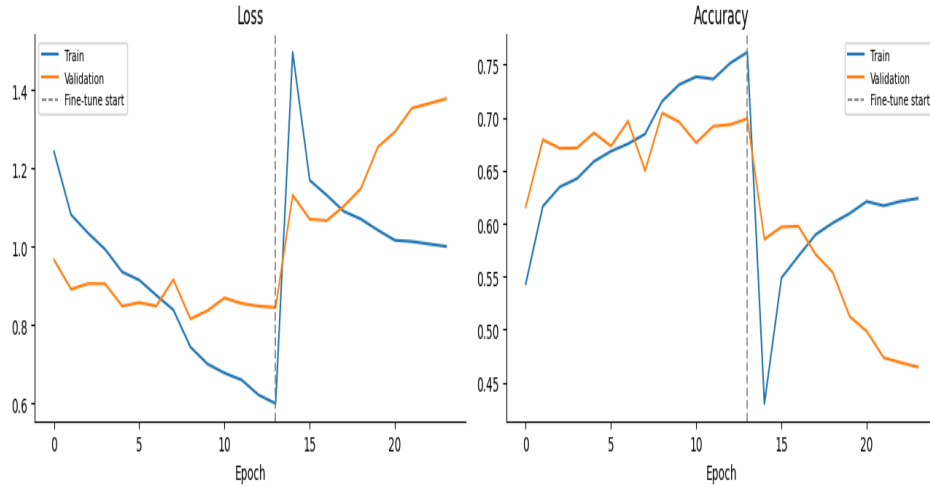


Figure 11. Two-phase MobileNetV2 learning curves. Phase 1 (frozen backbone) converges stably to 0.704 validation accuracy; Phase 2 fine-tuning on augmented data drops to 0.585 and degrades, indicating distribution shift.

Figures 12 and 13 compare the best model from each family on the test set. The SVM still leans on CMD, while the augmented CNN spreads its correct predictions more evenly and picks up the most ground on CGM and CBSD, the texture-sensitive classes where augmentation should help. The per-class F1 heatmap (Figure 14) pulls all of this together, and the one-vs-rest ROC curves (Figure 15) confirm the CNN's highest macro AUC (0.912). CBB stays the weakest class for every model.

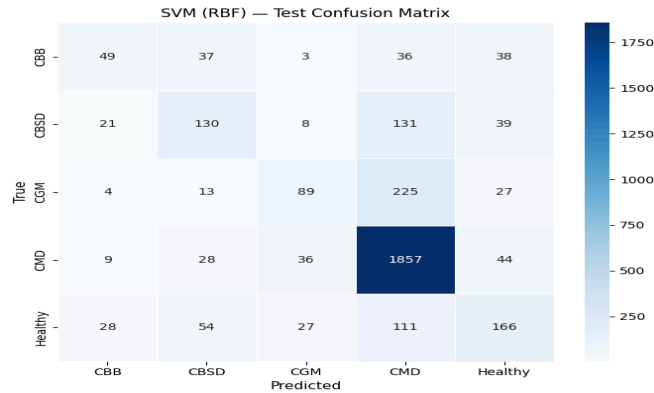


Figure 12. SVM (RBF) test confusion matrix. Strong CMD performance coexists with persistent CGM↔CMD confusion.

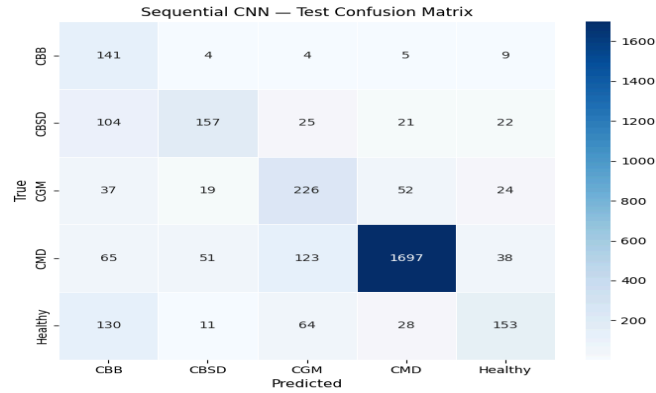


Figure 13. Augmented Sequential CNN test confusion matrix. Correct predictions are distributed more evenly across classes, with notable gains on CGM and CBSD relative to the SVM.

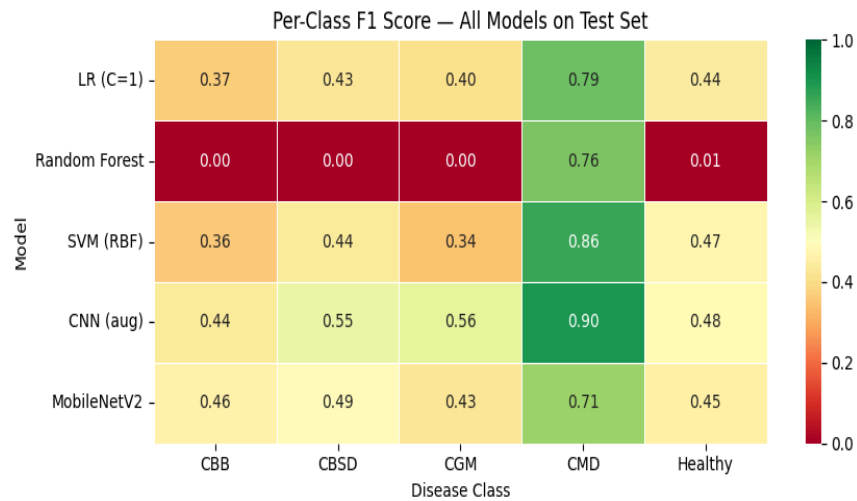


Figure 14. Per-class F1 heatmap across all five final models. CMD is uniformly strong; CBB and CGM are uniformly weak; the augmented CNN row is the most balanced overall.

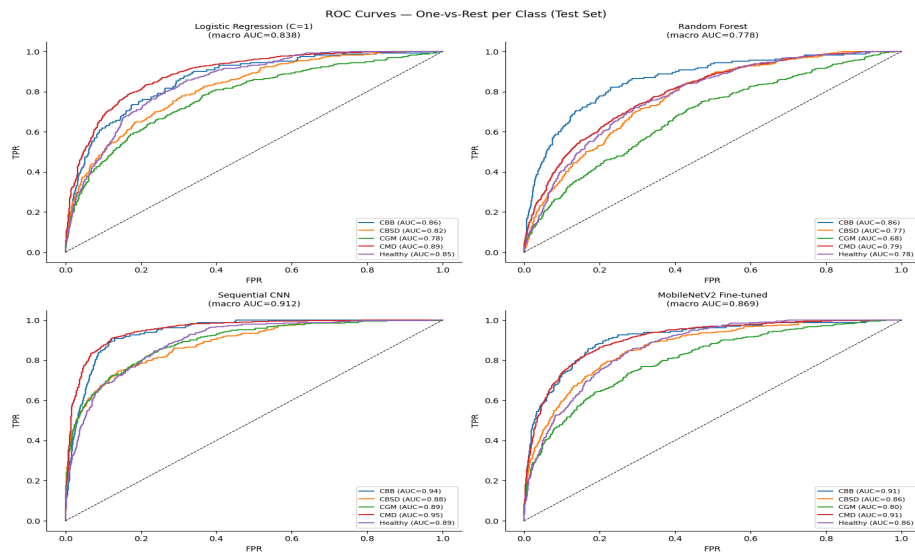


Figure 15. *One-vs-rest ROC curves per class on the test set, with macro AUC annotated per model. The augmented CNN attains the highest macro AUC (0.912); CBB shows the lowest separability across all models.*

6. Discussion

6.1 Linear Limits, the Random Forest Collapse, and the SVM Benchmark

Experiments 1 and 2 barely differ in validation macro-F1 (0.4743 vs 0.4735) and land on the same accuracy (0.608). Inside the 512 dimensional PCA space, then, the feature representation is the binding constraint, not the regularisation strength. Minority recall is moderate (CBB 0.42, CGM 0.51), so the linear boundary is catching some signal but runs into a wall wherever the class boundaries are nonlinear. That is consistent with the argument [7] that you need a kernel once the optimal boundary stops being a hyperplane.

The biggest failure in the study is Experiment 3. Random Forest gave CMD recall of 1.00 and almost nothing for everything else, landing at a macro-F1 of just 0.154 (Figure 7). The variance analysis explains why. HOG brings 6,084 dimensions but only 7.0% of the variance, and the colour histograms add 96 dimensions and roughly 0.0%. When PCA folds all 7,460 dimensions into 512 components, each component is a blend of features with wildly different value. Random Forest samples a random subset of those components at every split, and since most of them are HOG-dominated noise, most splits tell it nothing useful, so it falls back on the 61.5% CMD prior. The SVM avoids this trap because it works on the full feature space, and its RBF kernel finds a high dimensional mapping that separates the classes regardless of how good any single feature is. That is why Experiment 4 worked where Experiment 3 did not. The SVM was the best classical model (macro-F1 = 0.494, AUC = 0.868). Its confusion matrix (Figure 8) shows CMD (precision 0.80, recall 0.94) and Healthy handled best, while CGM and CBB (both recall 0.27) get confused with CMD, which makes sense given that all three involve generalised yellowing. The SVM's resilience under imbalance lines up with the observation [10] that support-vector classifiers handle imbalance better than ensembles, because the support vectors come disproportionately from the higher-weighted minority classes.

6.2 Augmentation, Overfitting, and Bias–Variance

The non augmented CNN (Figure 9) is a clean high variance case: low training loss next to high, oscillating validation loss. With no augmentation the model sees the same images every epoch, and the small CBB (763) and CGM (1,674) training sets are easy to memorise even with Dropout(0.5) and BatchNormalization in place. Augmentation (Figure 10) hits that variance directly by enlarging the effective training set, and the test numbers back it up: macro-F1 = 0.588 and AUC = 0.912, the best in the study. The gains land where you would hope, on CGM (0.565 vs the SVM's 0.342) and CBSD (0.550 vs 0.440), the classes where subtle texture differences carry the signal. That matches Shorten and Khoshgoftaar's finding [11] that augmentation helps most when classes look alike. The bias-variance picture across the whole set hangs together: Logistic Regression is high-bias, the SVM cuts bias with the kernel while the margin keeps variance in check, and the CNN goes from high variance without augmentation to well-regularised with it, all without touching the architecture or capacity.

6.3 The Unexpected Transfer Learning Result

The most interesting result, at least to me, is that fine-tuned MobileNetV2 (macro-F1 = 0.509) lost to the from-scratch CNN (0.588), even though the literature [5] says transfer learning usually wins on agricultural imaging with limited data. The transfer curves (Figure 11) pin down where it goes wrong. Phase 1, with a frozen backbone on clean images, converged steadily to 0.704, the most stable curve in the whole study. Phase 2 then did two things at once: it brought in augmented data and unfroze the top layers. Validation accuracy dropped straight to 0.585 and only crept back to 0.598 before early stopping. That is a distribution shift. The head had adapted to clean features of non-augmented leaves, and Phase 2 changed both the input distribution and the trainable weights at the same moment. The $1e-5$ rate was low enough to avoid catastrophic forgetting but too low for the top layers to re-adapt to augmented inputs before the validation plateau cut training off. The from scratch CNN never had this problem because it saw augmented data from epoch 1 and let the features and classifier settle together. The takeaway is that keeping the data regime consistent across phases can matter more than the head start pretraining gives you.

6.4 Error Patterns, Limitations, and Trade-offs

Figures 12 to 15 show the same error structure repeating. CBB is the hardest class everywhere, with test F1 running from 0.00 (Random Forest) up to 0.46 (MobileNetV2) and never clearing 0.44 for the CNN. With only 763 training images and a look that mimics healthy-leaf blemishes, even its ROC AUC trails the rest (Figure 15), so this is a real learning problem rather than a threshold that needs tuning. CMD is the opposite. At 61.5% prevalence it posts the highest per-class F1 everywhere (0.76 to 0.90) but pumps up accuracy far more than it does macro-F1, and the gap between accuracy (0.72 SVM, 0.74 CNN) and macro-F1 (0.49, 0.59) is exactly the reason I chose macro-averaging in the first place. A few limitations are worth stating plainly. Every image comes from Uganda, so I cannot say how the models would do elsewhere. I used a single split, so the 0.079-point gap between the CNN and MobileNetV2 has no confidence interval behind it; stratified cross validation would fix that. And the Random Forest failure is more diagnostic than fatal, since feeding the MobileNetV2 embeddings to the ensemble without PCA would probably have been competitive, but I did not test that here. On the practical trade-off, the SVM trains in minutes on the compressed features while the CNN needs GPU time per epoch, so the CNN earns its keep when the extra macro-F1 is worth the cost and the SVM makes a cheap fallback.

7. Conclusion

This project ran eight experiments across classical and deep learning families on a realistic 21,397 image cassava field dataset. The augmented Sequential CNN came out best (test macro-F1 = 0.588, AUC = 0.912), edging past fine tuned MobileNetV2 (0.509) because of a distribution shift between the two transfer learning phases. Among the classical models the RBF SVM led (0.494). Random Forest failed almost completely (0.154) because PCA diluted the signal, which the variance analysis pins on the fact that 93% of the raw variance comes from the MobileNetV2 embeddings alone. Past the ranking, three findings hold up: macro-F1 is a more honest metric than accuracy under imbalance, augmentation measurably improves generalisation with the architecture fixed, and transfer learning is not automatically better than training from scratch. For a farmer-facing deployment, a from-scratch CNN with strong augmentation is a viable and arguably better choice, as long as low-confidence CBB predictions get flagged for a human to check.

8. Future Work

A few extensions follow naturally from the limitations. Focal loss [10] could help CBB, where even the best model only reaches $F1 = 0.44$, by down-weighting the easy majority examples. Heavier backbones like EfficientNet or Vision Transformers might pull out richer features than MobileNetV2, though at a higher cost. Targeted oversampling or synthetic images for CBB and CGM would attack the data-poverty problem directly, and collecting field data from other regions would test whether any of this generalises beyond Uganda. Ensembling the SVM probabilities with the CNN softmax scores might cancel out some of their complementary errors, and stratified cross-validation would put confidence intervals on the comparison. Finally, I would want Grad-CAM visualisations to confirm the CNN is actually looking at lesions rather than background clutter before anyone deploys it.

References

- [1] FAO, Cassava: The Drought-Tolerant Crop, Food and Agriculture Organization of the United Nations, Rome, 2023.
- [2] A. Bellotti, L. Smith, and S. Lapointe, “Recent advances in cassava pest management,” *Annual Review of Entomology*, vol. 44, pp. 343–370, 1999.
- [3] E. Mwebaze and A. Owomugisha, “Machine learning for plant disease identification and detection,” in *Proc. Int. Conf. on Learning Representations (ICLR) Workshop*, 2016.
- [4] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, Sep. 2016.
- [5] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. P. Hughes, “Deep learning for image-based cassava disease detection,” *Frontiers in Plant Science*, vol. 8, p. 1852, Oct. 2017.
- [6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.
- [7] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, Sep. 1995.
- [8] V. Vapnik, *The Nature of Statistical Learning Theory*, Springer, New York, 1995.
- [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [10] M. Buda, A. Maki, and M. A. Mazurowski, “A systematic study of the class imbalance problem in convolutional neural networks,” *Neural Networks*, vol. 106, pp. 249–259, Oct. 2018.
- [11] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, no. 1, p. 60, Jul. 2019.
- [12] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” *arXiv preprint arXiv:1511.08060*, 2015.
- [13] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. 886–893, 2005.
- [14] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, Apr. 2018.
- [15] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, Feb. 2018.

Appendix A: Full Experiment Configuration

Table A1. Complete hyperparameter configuration for all eight experiments.

Exp	Model	Full Configuration
1	Logistic Regression	C=0.01, penalty=L2, solver=lbfgs, multinomial, class_weight=balanced, PCA=512, max_iter=1000
2	Logistic Regression	C=1.0, penalty=L2, solver=lbfgs, multinomial, class_weight=balanced, PCA=512, max_iter=1000
3	Random Forest	n_estimators=300, min_samples_leaf=2, class_weight=balanced_subsample, PCA=512
4	SVM (RBF)	kernel=rbf, C=10, gamma=scale, probability=True (Platt), class_weight=balanced, PCA=512
5	Sequential CNN	4 conv blocks (32/64/128/256), BN, MaxPool, GAP, Dense(256)+ReLU, Dropout(0.5), softmax; Adam 1e-3; no augmentation
6	Sequential CNN	Same architecture as Exp 5; Adam 1e-3; augmentation (flip/rotate20/zoom15/brightness/contrast); ReduceLROnPlateau; EarlyStopping(7)
7	MobileNetV2 head	ImageNet weights, backbone frozen, Dense(256)+Dropout(0.4)+softmax head; Adam 1e-3; clean images
8	MobileNetV2 FT	Top 30/154 layers unfrozen; Adam 1e-5; augmented training set; preprocessing x=inputs*2-1

Dataset splits (stratified): Train 14,977 (70%) / Validation 3,210 (15%) / Test 3,210 (15%). Class weights: CBB 3.936, CBSD 1.955, CGM 1.794, CMD 0.325, Healthy 1.660. Feature dimensions before PCA: colour histogram 96, HOG 6,084, MobileNetV2 GAP 1,280 (total 7,460); PCA retained 512 components (68.5% variance).

Appendix B: Notebook and Reproducibility

All code, outputs, figures, and the experiment log are available in the project notebook on GitHub. The notebook runs top to bottom on Google Colab with a GPU runtime; random seeds are fixed for the data split and model initialisation. The experiment log and per-model test results are exported to CSV (experiment_log.csv and test_results.csv), and all fifteen figures reproduced in this report are saved under the figures directory. Repository: <https://github.com/Samkwizera/Model-Training-and-Evaluation.git>